

Glossar

Anwendung

Eine *Anwendung* ist eine Software, die eine bestimmte Funktionalität bereitstellt. Eine *Anwendung* kann aus mehreren *Komponenten* bestehen, z. B. einem *Frontend*, einem *Backend* und einer *Datenbank*.

Framework

Ein *Framework* stellt eine Reihe von Funktionen bereit, die verwendet werden können, um eine *Anwendung* zu entwickeln.

Frontend

Das *Frontend* ist ein Teil einer *Anwendung*, mit dem der Nutzer interagiert, und ist die Schnittstelle zum *Backend*.

Backend

Das *Backend* ist ein Teil einer *Anwendung*, welche Daten verarbeitet und persistiert. Das Backend kommuniziert mit dem *Frontend* über eine *API*.

Datenbank

Eine *Datenbank* ist eine Sammlung von Daten, die in einer definierten Form gespeichert sind. Die *Datenbank* wird vom *Backend* verwendet, um Daten zu speichern und abzurufen.

Architektur

Eine *Architektur* ist eine Struktur, die verwendet wird, um eine *Anwendung* zu entwickeln. Eine *Architektur* definiert, wie die *Anwendung* entwickelt wird und wie die *Komponenten* der *Anwendung* zusammenarbeiten.

Client-Server-Architektur

Eine *Client-Server-Architektur* ist eine Software-*Architektur*, bei der die *Anwendung* in zwei Teile aufgeteilt ist: ein *Frontend* und ein *Backend*.

NX

NX ist eine flexible *Anwendung*, das die Entwicklung von *Frontend* und *Backend* in einer bestimmten Struktur (z. B. *monolithische Architektur*) erleichtert.

<https://nx.dev/>

Monolithische Architektur

Eine *monolithische Architektur* ist eine Software-Architektur, bei der alle *Komponenten* einer *Anwendung* gemeinsam verwaltet werden.

Monorepo

Ein *Monorepo* ist ein Software-Entwicklungsansatz, bei dem der gesamte Quellcode einer *Anwendung* gemeinsam verwaltet wird.

<https://nx.dev/concepts/more-concepts/why-monorepos>

Generator

Ein *Generator* ist ein Werkzeug, das verwendet wird, um Code-Abschnitte (*Scaffold*) zu erstellen.

<https://nx.dev/core-features/plugin-features/use-code-generators>

Scaffold

Ein *Scaffold* ist eine Vorlage, die die Struktur und den Code für ein bestimmtes *Feature* oder eine bestimmte Funktion bereitstellt. Es ist ein generischer Code, der von einem *Generator* erstellt wird.

Feature

Ein *Feature* ist eine Funktion, die in einer *Anwendung* implementiert ist. Ein *Feature* kann aus mehreren *Komponenten* bestehen.

Angular

Angular ist ein *Framework* für die Entwicklung von *SPA*-Webanwendungen. Es wird verwendet, um *Frontend*-Anwendungen zu entwickeln, die in einem Browser ausgeführt werden.

<https://angular.io/>

Komponenten

Komponenten sind Bausteine einer *Angular Anwendung*. *Komponenten* bestehen mindestens aus einer HTML-Datei oder einem HTML-Template, sowie einer TypeScript-Datei, welche die Logik der *Komponenten* enthält.

<https://angular.io/guide/architecture-components>

Services

Services sind Klassen mit einem spezifischen Zweck. Sie werden verwendet, um die Logik einer *Anwendung* zu kapseln und die Wiederverwendbarkeit von Code zu ermöglichen.

<https://angular.io/guide/architecture-services>)

Libraries

Libraries sind wiederverwendbare Code-Bibliotheken, die in einer *Anwendung* verwendet werden können. *Libraries* können aus *Komponenten*, *Services* bestehen und bilden einen Use Case ab.

<https://nx.dev/concepts/more-concepts/library-types>

Cypress

Cypress ist eine *Anwendung* für *E2E-Tests* des *Frontend*. Es wird verwendet, um die Funktionalität der zu entwickelnden *Anwendung* zu testen, indem es definierte *E2E-Tests* in einem Browser simuliert, *API* abfängt und beantwortet.

<https://www.cypress.io/>

CI/CD

CI/CD ist die Abkürzung für Continuous Integration und Continuous Delivery. Continuous Integration ist ein Entwicklungsansatz, bei dem Entwickler ihre Änderungen in einem gemeinsamen Repository zusammenführen. Continuous Delivery ist ein Entwicklungsansatz, bei dem die Software in kurzen Zyklen ausgeliefert wird. Die Software wird automatisch getestet und kann jederzeit ausgeliefert werden.

<https://resources.github.com/ci-cd/>

E2E-Tests

E2E-Tests sind Tests, welche die Funktionalität der *Anwendung* testen. Sie simulieren die Verwendung der *Anwendung* durch einen Nutzer für vollständige Anwendungsfälle (Beispiel: Event erstellen angefangen mit dem einloggen auf der Webseite).

API

Eine *API* ist eine Schnittstelle, die verwendet wird, um mit einer *Anwendung* zu kommunizieren. Eine *API* kann verwendet werden, um Daten von einer *Anwendung* abzurufen oder Daten in einer *Anwendung* zu speichern.

RESTful-Webservices

RESTful-Webservices sind Webservices, welche HTTP-Methoden verwenden um Ressourcen zu erstellen, zu lesen, zu aktualisieren oder zu löschen.

Kotlin

Kotlin ist eine Programmiersprache, die von JetBrains bereitgestellt wird. *Kotlin* ist eine objektorientierte Programmiersprache und unterstützt viele Konzepte der funktionalen Programmierung. Im Rahmen dieses Projekts wird Kotlin-Bytecode auf einer Plattform-spezifischen JVM (Java Virtual Machine) ausgeführt, allerdings unterstützt Kotlin auch die Entwicklung von Plattform-Nativen-Anwendungen und Web-Anwendungen (JavaScript und/oder WASM).

Spring Boot

Spring Boot ist ein *Framework* für die Entwicklung von *Kotlin*-Anwendungen. Es vereinfacht die Entwicklung von *Kotlin*-Anwendungen, indem es eine Reihe von Bibliotheken und Werkzeugen bereitstellt, die häufig in *Kotlin*-Anwendungen verwendet werden.

<https://spring.io/projects/spring-boot>

Gradle

Gradle ist ein Build-Management-Tool für *Kotlin*, Java und Scala Anwendungen. Es wird für Build-Lifecycle-Management verwendet (Abhängigkeiten einer *Anwendung* verwalten, die *Anwendung* bauen, *Anwendung* testen, etc.).

<https://gradle.org/>

H2

H2 ist eine relationale *Datenbank*, die in *Java* geschrieben ist. *H2* wird häufig für Testzwecke verwendet, da sie in den Speicher geladen werden kann und keine Installation erfordert.

<https://www.h2database.com/html/main.html>

TypeScript

TypeScript ist eine typisierte Programmiersprache, die von Microsoft angeboten wird. Sie ist eine objektorientierte Programmiersprache, die auf JavaScript aufbaut. *TypeScript* wird in *Angular*-Anwendungen verwendet.

SPA

SPA ist die Abkürzung für Single Page Application. Eine *SPA* ist eine *Anwendung*, die nur eine einzige HTML-Seite lädt und die Inhalte der Seite dynamisch

aktualisiert, wenn der Nutzer mit der *Anwendung* interagiert. Die Inhalte werden über eine *API* vom *Backend* abgerufen.

Swagger

Swagger ist ein *Framework*, das verwendet wird, um *RESTful-Webservices* zu dokumentieren. Es wird verwendet, um die *API* einer *Anwendung* zu dokumentieren, damit andere Entwickler (Beispielsweise *Frontend*-Entwickler) die *API* verwenden können.

<https://swagger.io/>

NPM

NPM ist die Abkürzung für Node Package Manager. Es ist ein Paketmanager für JavaScript, der verwendet wird, um Abhängigkeiten zu verwalten und JavaScript-Anwendungen zu bauen.

<https://www.npmjs.com/>

NGRX

NGRX ist ein *Framework* für die Entwicklung von *Frontend*-Anwendungen mit *Angular*. Es wird verwendet, um den Zustand einer *Anwendung* zu verwalten.

<https://ngrx.io/>

Tailwind CSS

Tailwind CSS ist ein *CSS-Framework*, das verwendet wird, um das *Frontend* einer *Anwendung* zu stylen. Die Styles werden über Klassen angewendet, die in HTML-Elementen verwendet werden.

<https://tailwindcss.com/>

GitHub

GitHub ist ein Online-Dienst, der verwendet wird, um Quellcode zu verwalten. Es wird verwendet, um den Quellcode einer *Anwendung* zu speichern und zu verwalten.

<https://github.com/>

Issue

Ein *Issue* ist ein Problem, das in einem Repository auf *GitHub* gemeldet wird. Es wird verwendet, um Probleme mit dem Quellcode einer *Anwendung* zu melden. Problem ist hierbei etwas freier zu verstehen als ein Fehler, es kann sich auch um eine Verbesserung handeln.

PlantUML

PlantUML ist eine *Anwendung*, das verwendet wird, um UML-Diagramme zu erstellen. Es wird verwendet, um die Architektur einer *Anwendung* zu dokumentieren.

<https://plantuml.com/>

Google Maps

Google Maps ist ein Kartendienst, der von Google bereitgestellt wird. Er kann verwendet werden, um Karten in einer *Anwendung* anzuzeigen.

<https://www.google.com/maps>

Persönliche Daten

Daten, die sich auf eine identifizierte oder identifizierbare natürliche Person beziehen sind *persönliche Daten*. Beispiele für *persönliche Daten* sind Name, Adresse, Telefonnummer, E-Mail-Adresse, IP-Adresse, Standortdaten und Online-Kennungen.